

Usine de drones : de l'implémentation naïve aux design patterns

Choix techniques, patterns retenus et écartés, problèmes rencontrés et solutions — étapes 1 à 3.

SUJET	Système d'assemblage de drones piloté par instructions texte
LANGAGE	C# (.NET SDK, multiplateforme)
ÉTAPES COUVERTES	1 · Implémentation naïve – 2 · Design patterns – 3 · Modules complémentaires
DÉPÔT	tags <code>etape-1</code> , <code>etape-2</code> , <code>etape-3</code>

Objectif et déroulement

Le projet consiste à écrire un programme qui pilote l'assemblage de drones dans une usine : suivi du stock de pièces, calcul des besoins d'une commande, génération des instructions d'assemblage, vérification et production. Le sujet impose une progression en trois temps : une implémentation naïve pour poser le problème, un refactoring en design patterns, puis l'ajout de modules complémentaires. Ce rapport suit ce découpage et explique, pour chaque étape, ce qui a été fait et pourquoi.

— ÉTAPE 1

Implémentation naïve et ses limites

La première version tient dans un unique `Program.cs` : des dictionnaires statiques pour le stock de pièces et de drones, un grand bloc `if / else` qui interprète chaque instruction utilisateur, et une méthode qui imprime en dur la séquence d'assemblage d'un drone à cinq pièces fixes.

Cette version répond au sujet mais montre vite ses limites : ajouter une instruction alourdit le même bloc de code déjà chargé ; `ADD_TEMPLATE` n'est qu'un message d'erreur en attendant l'étape suivante ; rien ne distingue le stock, la validation des commandes et la génération des instructions, ce qui rend chaque modification risquée pour le reste du programme. C'est précisément ce que l'étape 2 doit corriger.

Mise en place des design patterns

Quatre patterns ont été retenus, chacun en réponse à un problème concret rencontré dans le code de l'étape 1.

COMMAND

Problème : le dispatcher `if / else` centralisait toute la logique des six instructions utilisateur dans une seule méthode.

Solution : chaque instruction (`STOCKS` , `NEEDED_STOCKS` , `INSTRUCTIONS` , `VERIFY` , `PRODUCE` , `ADD_TEMPLATE` , puis `RECEIVE` et `GET_MOVEMENTS` à l'étape 3) devient une classe `ICommand` , enregistrée par son nom dans un `CommandRegistry` . Ajouter une instruction revient à écrire une classe de plus, sans toucher au dispatcher ni aux commandes existantes.

STRATEGY

Problème : classer un drone dans une ou plusieurs catégories (Aérien / Marin / Terrestre / Submersible) demande une règle différente par catégorie, combinables et évolutives indépendamment.

Solution : une classe par règle (`AerienRule` , `MarinRule` , `TerrestreRule` , `SubmersibleRule`) implémentant `ICategoryRule` , exécutées par un `DroneCategorizer` qui retourne toutes les catégories satisfaites. Chaque règle se lit et se teste seule.

BUILDER

Problème : la séquence d'instructions d'assemblage (`GET_OUT_STOCK` , `INSTALL` , `ASSEMBLE` ...) obéit à un ordre imposé par le sujet (générateur avant module principal, déplacement après la coque, système installé avant assemblage).

Solution : `DroneAssemblyBuilder` encapsule cet ordre derrière des méthodes chaînées. Son intérêt s'est confirmé à l'étape 3 : quand un drone a pu avoir plusieurs générateurs ou modules de déplacement, seul le Builder a dû apprendre à boucler sur une liste — aucun autre fichier n'a eu besoin de connaître l'ordre d'assemblage.

FACTORY

Problème : transformer une liste de noms de pièces en un `DroneTemplate` valide demande d'identifier le rôle de chaque pièce, de vérifier les règles de construction et de calculer les catégories — une logique que les quatre drones initiaux et `ADD_TEMPLATE` doivent partager à l'identique.

Solution : `DroneTemplateFactory` centralise cette construction et cette validation, utilisée aussi bien à l'initialisation du programme que pour chaque appel utilisateur à `ADD_TEMPLATE`.

Patterns envisagés puis écartés

- **Singleton** pour le stock : écarté au profit d'une seule instance de `StockRepository` créée dans `Program.cs` et transmise explicitement aux commandes qui en ont besoin. Même garantie d'unicité, sans l'état global qui rend un Singleton difficile à isoler pour des tests.
- **Abstract Factory** : écarté au profit d'une Factory simple, car le projet ne construit qu'une seule famille de produits (des `DroneTemplate`), pas plusieurs familles cohérentes à faire varier ensemble.
- **Observer** pour la traçabilité des flux (étape 3) : envisagé pour notifier un journal à chaque mouvement de stock, puis écarté — un seul type d'abonné (le journal lui-même) ne justifiait pas l'indirection d'un vrai pattern Observer ; `StockRepository` enregistre directement chaque mouvement.

4.2 – Catégorisation des drones

Chaque catégorie combine des contraintes sur la coque, le système installé et les modules de déplacement. Deux points du sujet n'étaient pas explicites et ont demandé

un choix documenté : la règle « un module de déplacement (X) » est satisfaite dès qu'*au moins un* module de déplacement porte le tag X (utile une fois les drones à plusieurs modules autorisés en étape 3) ; et pour Submersible, « toutes les pièces (S) » ne peut porter que sur la coque, les générateurs et les modules de déplacement, car ce sont les seules pièces du catalogue à porter la dimension F/M/L/S (le module principal, le système et le module de contrôle portent une dimension 2D/3D distincte).

4.3 – ADD_TEMPLATE

Le sujet ne précise pas si le système principal fait partie de la liste de pièces d' `ADD_TEMPLATE` . Le choix retenu est de l'y inclure au même titre que la coque ou le générateur : le rôle de chaque pièce est déduit automatiquement du catalogue, ce qui évite d'introduire une syntaxe différente pour ce seul cas.

Modules complémentaires

Deux modules ont été choisis parmi ceux proposés, auxquels s'ajoutent deux prérequis transverses imposés par le sujet pour cette phase.

ÉLÉMENT	STATUT	DESCRIPTION
5.1.1	Prérequis	Réception de stock — <code>RECEIVE ARGS</code>
5.1.2	Prérequis	Jusqu'à 2 générateurs / 3 modules de déplacement, avec règle de couplage
5.2.1	Module choisi	Modification de drones — <code>WITH</code> / <code>WITHOUT</code> / <code>REPLACE</code>
5.2.3	Module choisi	Traçabilité des flux — <code>GET_MOVEMENTS</code>

Le prérequis 5.1.2 s'est avéré être une dépendance technique du module de modification : `WITH` / `WITHOUT` pouvant ajouter ou retirer des générateurs ou des modules de déplacement, `DroneTemplate` devait d'abord accepter plusieurs pièces par rôle avant que ces instructions aient un sens à valider. Les deux ont donc été traités dans la même passe : `ConstructionRules` centralise la règle (1 à 2 générateurs, 1 à 3 modules de déplacement, 2 générateurs obligatoires dès 2 modules de déplacement) et est réutilisée aussi bien par `DroneTemplateFactory` que par le calcul des modifications.

Pour `WITH` / `WITHOUT` / `REPLACE`, le séparateur passe de la virgule au point-virgule dès qu'un de ces mots-clés apparaît dans la commande — l'ancien format reste accepté tel quel en leur absence. Les modifications s'appliquent une fois par ligne de commande, affectent `NEEDED_STOCKS`, `VERIFY` et `PRODUCE`, et déclenchent une revalidation de la règle de construction dès qu'elles touchent un générateur ou un module de déplacement. Le sujet ne fournissant aucun exemple de sortie `INSTRUCTIONS` avec modificateurs, cette commande continue d'imprimer la séquence du template de base — limitation assumée et documentée plutôt que comblée par une supposition non vérifiable.

La traçabilité repose sur un principe simple : chaque méthode de `StockRepository` qui modifie le stock (`RemovePieces`, `AddPiece`, `AddDrones`) enregistre un mouvement

horodaté avec un motif (`PRODUCE` , `RECEIVE`). `GET_MOVEMENTS` lit simplement ce journal, en entier ou filtré sur une liste d'éléments.

Problèmes rencontrés et solutions

01 Build incompatible avec la machine de développement

CONSTAT

Le script de build initial appelait `csc.exe` de .NET Framework via un chemin Windows (`$env:WINDIR`), inutilisable sur macOS.

SOLUTION

Migration vers un projet SDK `dotnet` standard (`.csproj`), multiplateforme, qui compile automatiquement tous les fichiers du dossier — un prérequis de toute façon nécessaire pour répartir le code en plusieurs classes.

02 Version du runtime .NET disponible

CONSTAT

Seul le runtime .NET 10 était installé localement, pas de .NET 8 (LTS).

SOLUTION

Cible ajustée à `net10.0` pour permettre la compilation et l'exécution locales ; point à revérifier sur la machine utilisée pour la notation.

03 Dépôt git mal isolé

CONSTAT

Le dossier de travail se trouvait initialement à l'intérieur d'un dépôt git racine au niveau du dossier utilisateur, mêlant ce projet à d'autres dossiers sans rapport.

SOLUTION

Un dépôt git indépendant a été initialisé directement dans le dossier du projet, avec son propre remote, pour ne versionner que les fichiers du projet.

04 Points du sujet non spécifiés

CONSTAT

Plusieurs comportements ne sont pas explicitement décrits par le sujet : contenu exact des arguments d' `ADD_TEMPLATE` , portée de `WITH` / `WITHOUT` sur `INSTRUCTIONS` , format de sortie de `GET_MOVEMENTS` , interprétation de « toutes les pièces (S) » pour la catégorie Submersible.

Plutôt que deviner silencieusement, chaque choix a été explicité dans le fichier `README.md` du projet (section « Hypothèses de cette version »), pour rester justifiable en soutenance et permettre de revenir dessus si besoin.

— CONCLUSION

Bilan

Le projet couvre les trois phases demandées : une implémentation naïve fonctionnelle, un refactoring en quatre design patterns justifiés (Command, Strategy, Builder, Factory) avec catégorisation des drones et templates dynamiques, puis deux modules complémentaires (modification de drones, traçabilité des flux) construits sur les prérequis de contraintes de construction et de réception de stock. Chaque étape a été vérifiée manuellement en rejouant les scénarios du sujet et des cas limites, et le code compile sans erreur à chaque étape taguée du dépôt.